

Goal Stack Planning

What to say during the presentation, slide by slide. Chapter 4: Planning, Introduction to Artificial Intelligence (3171105). Presenter: Prashant Kushwaha

Slide 1: Introduction

Good morning everyone. My name is Prashant Kushwaha, and today I will explain Goal Stack Planning, which is a topic from Chapter 4, Planning, of our subject Introduction to Artificial Intelligence.

Let me first tell you what we will cover. We will see what goal stack planning is, the blocks world problem and its operators, the algorithm step by step, one solved example, and finally its limitations.

Slide 2: What is goal stack planning?

First, what is planning? Planning means finding a sequence of actions that takes us from an initial state to a goal state. For example, if a robot has to arrange some blocks in a particular order, the plan tells the robot which action to do first, which second, and so on.

Goal stack planning is one of the oldest and simplest ways of doing this. It was used in a planner called STRIPS. The main idea is that the planner keeps a stack, and at every step it looks only at the item on top of that stack. That item can be a goal that we want to achieve, or an operator that we want to apply.

There are three things to remember. The first is the world state, which is just a list of facts that are currently true, for example that block B is on block A, or that the robot arm is empty. The second is the goal stack itself, which stores the goals and operators still waiting to be handled. The third is the plan, which is the list of operators we have actually applied, in order. When the stack becomes empty, that list is our final answer.

Because this method solves one subgoal after another, we call it a linear planner. Keep this word in mind, because it explains the limitation we will see later.

Slide 3: The blocks world and its operators

To understand goal stack planning, the textbooks use a problem called the blocks world. Imagine a table, some blocks placed on it, and a robot arm. The arm can hold only one block at a time, and it can only move a block that has nothing on top of it.

We describe this world using five facts. $ON(x,y)$ means block x is on block y . $ONTABLE(x)$ means x is on the table. $CLEAR(x)$ means nothing is on top of x . $HOLDING(x)$ means the arm is holding x . And $ARMEMPTY$ means the arm is holding nothing.

There are four operators, or actions. `STACK(x,y)` puts the block we are holding on top of `y`. `UNSTACK(x,y)` takes `x` off from `y` and holds it. `PICKUP(x)` lifts a block from the table, and `PUTDOWN(x)` places the held block back on the table.

Each operator has three parts. The precondition tells us what must be true before we are allowed to use the operator. The delete list tells us which facts become false after using it. And the add list tells us which facts become true. For example, to `STACK x` on `y`, the arm must be holding `x` and `y` must be clear. After stacking, the arm becomes empty and `x` is on `y`, while `HOLDING(x)` and `CLEAR(y)` are no longer true.

Slide 4: How the algorithm works

Now let us see the algorithm. We start by pushing the goal on the stack, and then we repeat one simple loop until the stack is empty. In each round we look at the top item, and there are four possible cases.

Case one: the top is a compound goal, meaning several conditions joined by AND. We push it back, and then we push each of its parts on top of it, in any order. The compound goal is kept underneath so that when all its parts are done, we can check the whole goal once more.

Case two: the top is a single goal that is already true in the current state. Then there is nothing to do, so we simply pop it.

Case three: the top is a single goal that is not yet true. Then we choose an operator whose add list contains this goal, because that operator can make it true. We push the operator, and on top of it we push the operator's preconditions, so that the preconditions get solved first.

Case four: the top is an operator. This means all its preconditions have already been satisfied. So we pop it, update the state using its delete list and add list, and write it into the plan.

That is the whole algorithm. It only ever does these four things.

Slide 5: Worked example

Now let me show this with an example. In the start state, block `A` is on the table with block `B` on top of it. Blocks `C` and `D` are separately on the table, and the arm is empty. Our goal is to have `C` on `A`, `B` on `D`, and the arm empty.

Step one. We push the goal and its parts. The planner picks `ON(C,A)` first. To achieve it we need `STACK(C,A)`, and its preconditions are `CLEAR(A)` and `HOLDING(C)`.

Step two. `CLEAR(A)` is not true, because `B` is sitting on `A`. So we use `UNSTACK(B,A)`. Its preconditions are all true, so we apply it. Now the arm is holding `B`.

Step three. Next we need HOLDING(C), which comes from PICKUP(C). But PICKUP needs the arm to be empty, and right now the arm is holding B. To make the arm empty we use PUTDOWN(B), and then we can do PICKUP(C).

Step four. Now both preconditions of STACK(C,A) are true, so we apply STACK(C,A). The first part of the goal, ON(C,A), is done.

Step five. We move to the second part, ON(B,D). For that we do PICKUP(B) and then STACK(B,D). When the stack is empty, the whole goal is satisfied.

So our final plan is: UNSTACK B from A, PUTDOWN B, PICKUP C, STACK C on A, PICKUP B, and STACK B on D. That is six steps.

Slide 6: Limitations

Goal stack planning has some good points. It is simple to implement, easy to trace by hand, and every operator in the plan is there because some goal needed it. It works well when the subgoals do not disturb each other.

But it also has weaknesses. First, the plan may be longer than needed. In our example, instead of putting B down on the table, we could have stacked B directly on D. Then the plan would be only four steps: UNSTACK B from A, STACK B on D, PICKUP C, STACK C on A. Our planner did not see this, because it handles one subgoal at a time.

The second and bigger problem is goal interaction, and the famous example is the Sussman anomaly. Here C is on A, and B is on the table, and the goal is A on B and B on C. If the planner finishes A on B first, it must later break that arrangement to put B on C. If it does B on C first, it must break that to put A on B. So whichever goal it does first, finishing the second goal undoes the first one, and the plan becomes wasteful.

The solution is to allow the planner to mix the steps of different subgoals instead of finishing them one by one. This is what plan space planning and hierarchical planning do, and these are the next topics in this chapter.

Slide 7: Summary and conclusion

To summarise. Goal stack planning uses one stack that holds goals and operators, and the item on top decides the next move. When a goal is not true, we replace it with an operator that achieves it and push the preconditions above it. Every operator we apply goes into the plan. Because it solves subgoals in a fixed linear order, it can give longer plans and fails on problems like the Sussman anomaly.

For this topic I have referred to the books in our syllabus: Rich and Knight, Khemani, Russell and Norvig, and Barr and Feigenbaum's Handbook of Artificial Intelligence, along with some

online material on STRIPS and the Sussman anomaly.

Thank you for listening. I am happy to take any questions.